N Reynas

Cervantes Araujo Carlos Iván / Manjarrez Olivas Mariana

Ingeniería en Sistemas Computacionales

Tópicos de Inteligencia Artificial

Instituto Tecnológico de Culiacán

9 de octubre de 2026

Problema de las N-Reinas con Búsqueda Tabú

1. Descripción del problema e introducción

El problema de las 8 reinas consiste en colocar 8 reinas en un tablero de ajedrez de 8 × 8 sin que ninguna ataque a otra. Dos reinas se atacan si comparten fila, columna o diagonal.

El espacio de búsqueda es grande: con una reina por columna hay 8^8 = 16 777 216 configuraciones posibles, y solo 92 son soluciones. Por eso se usa una búsqueda local en lugar de revisar todas las combinaciones.

La búsqueda local simple (ascenso de colinas) se queda atascada en óptimos locales: estados donde ningún vecino mejora. La **Búsqueda Tabú**, propuesta por Fred Glover, resuelve esto con una memoria de corto plazo. Permite moverse a vecinos peores y prohíbe por un tiempo deshacer los movimientos recientes, para no regresar a los mismos estados.

Este documento describe la representación del problema, el algoritmo, su implementación en Python con métricas de tiempo y movimientos, y las reglas de prueba.

2. Representación de P y S

P (problema) se define por sus elementos de búsqueda:

Elemento	Definición en el código
Estado	Lista de 8 enteros: `estado[columna] = fila` de la reina en esa columna
Estado inicial	Aleatorio: `[random.randrange(8) for _ in range(8)]`
Función objetivo h	Número de pares de reinas que se atacan (misma fila o misma diagonal)
Meta	h = 0
Vecindario	Mover una reina a otra fila de su misma columna: 8 × 7 = 56 vecinos
Lista tabú	Pares (columna, fila) que una reina acaba de dejar; tenencia = 10
Criterio de aspiración	Un movimiento tabú se permite si da un h menor que el mejor encontrado
Límite	1000 movimientos como máximo

Al poner una reina por columna, los ataques por columna quedan eliminados desde el inicio. Solo se cuentan los de fila y diagonal:

```
h(e) = \sum_{i<j} \left[\, e_i = e_j \;\lor\; |e_i - e_j| = j - i \,\right]
```

S (solución) es un estado con h = 0. Ejemplo obtenido al ejecutar el programa:

```
S = [3, 1, 6, 2, 5, 7, 4, 0]   h = 0

. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . Q . . . . .
. . . . . Q . .
```

3. Propuesta de algoritmo (pseudocódigo)

En cada paso el algoritmo evalúa los 56 vecinos, elige el mejor que no sea tabú (o que cumpla aspiración) y se mueve aunque empeore. Termina al encontrar h = 0 o al llegar a 1000 movimientos.

```
ALGORITMO BusquedaTabu(estado)
    inicio ← reloj()
    mejor ← estado
    tabu ← lista vacía
    movimientos ← 0

    MIENTRAS h(mejor) > 0 Y movimientos < MAX_MOV HACER
        mejorVecino ← nulo ;  mejorH ← ∞
        PARA cada columna c DE 0 A N-1 HACER
            PARA cada fila f DE 0 A N-1, f ≠ estado[c] HACER
                vecino ← estado con la reina de c movida a la fila f
                SI (c, f) ∈ tabu Y h(vecino) ≥ h(mejor) ENTONCES
                    continuar            // tabú sin aspiración
                SI h(vecino) < mejorH (empate: elegir al azar) ENTONCES
                    mejorVecino ← vecino ;  mejorH ← h(vecino)
                    mov ← (c, estado[c])  // fila que se abandona
        FIN PARA

        estado ← mejorVecino
        movimientos ← movimientos + 1
        agregar mov a tabu
        SI |tabu| > TENENCIA ENTONCES quitar el más antiguo
        SI mejorH < h(mejor) ENTONCES mejor ← estado
    FIN MIENTRAS

    tiempo ← reloj() - inicio
    DEVOLVER mejor, movimientos, tiempo
```

Parámetros: N = 8, TENENCIA = 10, MAX_MOV = 1000.

4. Implementación en Python

El programa `reinas_tabu.py` ya incluye las dos métricas pedidas:

- **Tiempo**: se mide con `time.perf_counter()` al inicio y al final de `busqueda_tabu`, y se reporta en milisegundos.

- **Cantidad de movimientos**: el contador `movimientos` sube en 1 cada vez que el algoritmo cambia de estado.

```
"""
Problema de las 8 reinas con Búsqueda Tabú.

Estado: lista de 8 números, estado[columna] = fila de la reina.
h(estado): número de pares de reinas que se atacan. Meta: h = 0.
Vecino: mover una reina a otra fila de su misma columna (56 vecinos).
Lista tabú: (columna, fila) que una reina acaba de dejar.
"""

import random
import time

N = 8
TENENCIA = 10      # tamaño de la lista tabú
MAX_MOV = 1000     # máximo de movimientos


def ataques(estado):
    h = 0
    for i in range(N):
        for j in range(i + 1, N):
            if estado[i] == estado[j] or abs(estado[i] - estado[j]) == j - i:
                h += 1
    return h


def busqueda_tabu(estado):
    inicio = time.perf_counter()
    mejor = estado[:]
    tabu = []
    movimientos = 0

    while ataques(mejor) > 0 and movimientos < MAX_MOV:
        # Buscar el mejor vecino permitido
        mejor_vecino, mejor_h, mov = None, None, None
        for col in range(N):
            for fila in range(N):
                if fila == estado[col]:
                    continue
                vecino = estado[:]
                vecino[col] = fila
                h = ataques(vecino)
                # Un movimiento tabú solo se permite si mejora al mejor (aspiración)
                if (col, fila) in tabu and h >= ataques(mejor):
                    continue
                if mejor_h is None or h < mejor_h or (h == mejor_h and
random.random() < 0.5):
                    mejor_vecino, mejor_h, mov = vecino, h, (col, estado[col])

        # Moverse al vecino y prohibir regresar
        estado = mejor_vecino
        movimientos += 1
```

```
        tabu.append(mov)
        if len(tabu) > TENENCIA:
            tabu.pop(0)

        if mejor_h < ataques(mejor):
            mejor = estado[:]

    tiempo = time.perf_counter() - inicio
    return mejor, movimientos, tiempo


def imprimir(estado):
    for fila in range(N):
        print(" ".join("Q" if estado[col] == fila else "." for col in range(N)))


def rendimiento(pruebas=100):
    exitos, total_mov, total_t = 0, 0, 0
    for _ in range(pruebas):
        sol, mov, t = busqueda_tabu([random.randrange(N) for _ in range(N)])
        exitos += ataques(sol) == 0
        total_mov += mov
        total_t += t
    print(f"\nRendimiento con {pruebas} estados aleatorios")
    print(f"Éxitos: {exitos}/{pruebas}")
    print(f"Movimientos promedio: {total_mov / pruebas:.2f}")
    print(f"Tiempo promedio: {total_t / pruebas * 1000:.3f} ms")


if __name__ == "__main__":
    inicial = [random.randrange(N) for _ in range(N)]
    print("Estado inicial:", inicial, "h =", ataques(inicial))
    imprimir(inicial)

    solucion, movimientos, tiempo = busqueda_tabu(inicial)

    print("\nSolución:", solucion, "h =", ataques(solucion))
    imprimir(solucion)
    print(f"\nMovimientos: {movimientos}")
    print(f"Tiempo: {tiempo * 1000:.3f} ms")

    rendimiento(100)
```

5. Reglas y resultados de prueba

En una ejecución de prueba, el algoritmo resolvió 100 de 100 estados aleatorios, con 13.22 movimientos y 3.612 ms en promedio.

Reglas

1. **Estado aleatorio**: cada prueba parte de un tablero generado con `random.randrange(8)` para cada columna. No se usa un estado fijo.
2. **Ejercicio de rendimiento en clase**: la función `rendimiento(pruebas)` corre el algoritmo sobre varios estados aleatorios y reporta éxitos, movimientos promedio y tiempo promedio.

Ejecución individual (estado inicial [6, 1, 1, 5, 1, 7, 6, 0], h = 7):

Métrica	Valor
Solución	[3, 1, 6, 2, 5, 7, 4, 0], h = 0
Movimientos	5
Tiempo	0.906 ms

Rendimiento con 100 estados aleatorios:

Métrica	Valor
Éxitos	100 / 100
Movimientos promedio	13.22
Tiempo promedio	3.612 ms

Los valores cambian en cada ejecución porque el estado inicial y los desempates son aleatorios. El tiempo también depende de la computadora.

Referencias

1. Glover, F. y Melián, B. (2003). Búsqueda Tabú. *Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial*, 7(19), 29–48.
2. Melián, B., Moreno Pérez, J. A. y Moreno Vega, J. M. (2003). Metaheurísticas: una visión global. *Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial*, 7(19), 7–28.
3. Russell, S. y Norvig, P. (2004). *Inteligencia Artificial: un enfoque moderno* (2.ª ed.). Madrid: Pearson Prentice Hall. Capítulo 4: búsqueda local y el problema de las 8 reinas.
4. Nilsson, N. J. (2001). *Inteligencia artificial: una nueva síntesis*. Madrid: McGraw-Hill.
5. Python Software Foundation. *time — Acceso a tiempo y conversiones*. Documentación de Python 3 en español. https://docs.python.org/es/3/library/time.html